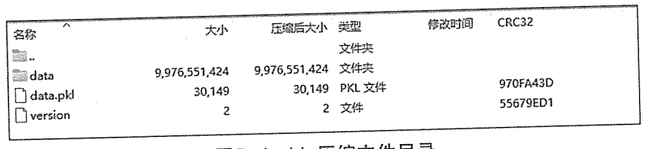
Hugging Face 模型目录中 bin 文件实际是个 Zip 压缩文件，以 llama-2-7b-hf 目录中的 pytorch_model-00001-of-00002.bin 为例，用解压缩程序打开看到的目录见图 5-4。

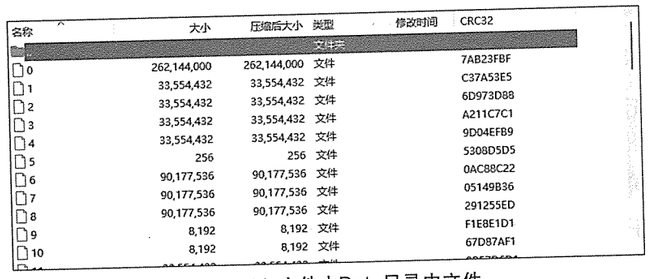


名称	大小	压缩后大小	类型	修改时间	CRC32
..			文件夹		
data	9,976,551,424	9,976,551,424	文件夹		
data.pkl	30,149	30,149	PKL 文件		970FA43D
version	2	2	文件		55679ED1

图5-4　bin压缩文件目录

Data.pkl 文件是个经过 Python 序列化文件，用于保存参数，读取它必须经过反序列化处理。

Data 目录中，保存有模型每个部分的权重，比如 q_proj。每个部分保存一个独立文件，具体见图 5-5。



名称	大小	压缩后大小	类型	修改时间	CRC32
			文件夹		
0	262,144,000	262,144,000	文件		7AB23FBF
1	33,554,432	33,554,432	文件		C37A53E5
2	33,554,432	33,554,432	文件		6D973D88
3	33,554,432	33,554,432	文件		A211C7C1
4	33,554,432	33,554,432	文件		9D04EFB9
5	256	256	文件		5308D5D5
6	90,177,536	90,177,536	文件		0AC88C22
7	90,177,536	90,177,536	文件		05149B36
8	90,177,536	90,177,536	文件		291255ED
9	8,192	8,192	文件		F1E8E1D1
10	8,192	8,192	文件		67D87AF1

图5-5　bin文件中Data目录中文件

共 323 个文件，其中 pytorch_model-00001-of-00002.bin 保存 241 个，pytorch_model-00002-of-00002.bin 保存 82 个。与 pytorch_model.bin.index.json 文件中权重行数对应：

```
"lm_head.weight": "pytorch_model-00002-of-00002.bin",
"model.embed_tokens.weight": "pytorch_model-00001-of-00002.bin",
"model.layers.0.input_layernorm.weight": "pytorch_model-00001-of-00002.bin",
"model.layers.0.mlp.down_proj.weight": "pytorch_model-00001-of-00002.bin",
"model.layers.0.mlp.gate_proj.weight": "pytorch_model-00001-of-00002.bin",
"model.layers.0.mlp.up_proj.weight": "pytorch_model-00001-of-00002.bin",
"model.layers.0.post_attention_layernorm.weight": "pytorch_model-00001-of-00002.bin",
"model.layers.0.self_attn.k_proj.weight": "pytorch_model-00001-of-00002.bin",
"model.layers.0.self_attn.o_proj.weight": "pytorch_model-00001-of-00002.bin",
"model.layers.0.self_attn.q_proj.weight": "pytorch_model-00001-of-00002.bin",
"model.layers.0.self_attn.rotary_emb.inv_freq": "pytorch_model-00001-of-00002.bin",
"model.layers.0.self_attn.v_proj.weight": "pytorch_model-00001-of-00002.bin",
"model.layers.1.input_layernorm.weight": "pytorch_model-00001-of-00002.bin",
"model.layers.1.mlp.down_proj.weight": "pytorch_model-00001-of-00002.bin",
......
```

文件大小和权重参数对应，比如文件 0 对应 lm_head.weight，字节数为 262144000，而 lm_head.weight 的形状为 [32000,4096]，数据类型为 F16，2 个字节，$32000 \times 4096 \times 2 = 262,144,000$。

2. pickle.Unpickler 反序列化

读取权重文件中的 Data.pkl 参数文件，需要了解 Python 的反序列化操作。

Python 的 pickle 模块实现了强大的序列化和反序列化功能，也就是说，它可以将 Python 对象转换为字节流（序列化），也可以将这些字节流重新构建为 Python 对象（反序列化）。这对于数据持久化和传输非常有用。

以下是 pickle 模块的四个主要接口。

pickle.dumps(obj)：将一个 Python 对象序列化为一个字节流。

pickle.loads(bytes_object)：将一个字节流反序列化为一个 Python 对象。

pickle.dump(obj, file)：将一个 Python 对象序列化并保存到一个文件中。

pickle.load(file)：从一个文件中读取字节流，并反序列化为一个 Python 对象。

可以简单用表 5-8 来分类。

表 5-8　pickle 模块的接口分类

操作	序列化	反序列化
转换 + 文件操作	dump()	load()
转换	dumps()	loads()

反序列化是指将序列化后的字节流恢复为原始的 Python 对象。pickle 库可以从字节流中恢复出原始的对象，使我们重新使用这些对象进行操作。如果反序列化需求比较简单，你可以直接使用 pickle.load 函数。如果需要更大的控制和灵活性，可以使用 pickle.Unpickler 类，创建一个 Unpickler 对象，并使用其 load 方法来反序列化数据。此外，还可以覆盖 Unpickler 的某些方法（例如 persistent_load 和 find_class），以处理更为复杂的反序列化需求，例如处理持久化对象或自定义类的加载方式。

以下是 pickle.Unpickler 的一些主要方法。

（1）**load()**：这个方法从打开的文件或者类似文件的对象（必须以二进制模式打开）中读取一个值，并返回它。如果到达文件的末尾，将抛出 EOFError。在反序列化过程中，如果遇到了一个持久化的对象 ID（身份标识号），会调用 persistent_load 方法。

（2）**persistent_load(pid)**：这是一个方法，可以在你的 Unpickler 子类中进行覆盖，以提供处理持久化对象 ID 的方式。在 pickle 模块中，这个方法并没有定义，如果你没有覆盖这个方法，而又在反序列化过程中遇到一个持久化的对象 ID，将会抛出一个 pickle.UnpicklingError。pid 是持久化对象 ID。

（3）**find_class(module, name)**：这是一个方法，可以在你的 Unpickler 子类中进行覆盖，以提供处理从 pickle 数据中加载类的方式。默认情况下，这个方法会从指定的模块中查找并返回指定的类。如果这个类不存在，或者模块不能被导入，将会抛出一个 AttributeError 或者 ImportError。

3. 源代码分析

每个 HF 模型目录有多个 bin 文件，下面代码每次读一个。代码中主要工作是反序列化，核心是 LazyUnpickler 类，它继承了 pickle.Unpickler 类，重写了类的 persistent_load 函数。代码通过解析 bin 压缩包中的 Data.pkl 文件，读出模型的组成部分，以及每个部分对应的形状和数据类型，最后计算出总参数量和字节数。

代码中比较难理解的是 LazyUnpickler 类中的 pid 和 find_class 函数中的 module 参数。实际上这两个变量是在序列化时预先定义好的。如果不知道预先定义内容，是无法做反序列化处理的。